

Lab Report: Week 2 Session B

Context Engineering with AGENTS.md

Mahmudul Hasan (Student ID: 4125999049)

May 17, 2026

Abstract

This report documents Week 2 Session B on **context engineering** using `AGENTS.md` with OpenCode Desktop. Exercise 1 generated project context from the real urban flood-warning codebase in `week2.session.a/` (FastAPI, Open-Meteo) rather than the lab template (Streamlit, OpenWeather). Exercise 2 compared the same prompt *Write a function to fetch weather data* without project context (empty folder, sync `requests`, dict) versus with `AGENTS.md` (async `aiohttp`, `WeatherData`, multi-zone fetch). Evidence: three OpenCode screenshots and the prompt log in Appendix A.

1 Introduction

Large language models load only a limited context per chat. `AGENTS.md` is a project-level instruction file that tells coding agents which stack, layout, and conventions to follow. This session practiced creating, testing, and refining that file for a rainfall / flood-monitoring project developed in Week 2 Session A.

2 Objectives

- Create an effective `AGENTS.md` for a rainfall monitoring project.
- Understand how context shapes AI-generated code.
- Compare outputs with and without `AGENTS.md`.
- Build a reusable prompting and documentation workflow.

3 Methods

3.1 Exercise 1: Create AGENTS.md

The lab prompt requested Streamlit, OpenWeather, and pandas. OpenCode was run with the `ai.water.lab` tree open. The agent scanned `week2.session.a/` and wrote `week2.session.b/AGENTS.md` aligned to the **actual** stack: FastAPI, Open-Meteo, TimescaleDB, Redis, zone-based flood model, and alert routing. Figure 1 shows the session.

3.2 Exercise 2: Test effectiveness

The same user message was used twice:

Write a function to fetch weather data

- **Without context:** new chat in an **empty folder** (no `AGENTS.md`, no repo files). The agent asked for language and path, then produced `fetch_weather_demo.py` after a follow-up specification.

- **With context:** project open with `AGENTS.md` and existing code; the agent created `weather_fetcher.py` using async Open-Meteo access and project conventions.

3.3 Exercise 3: Refine AGENTS.md

After testing, the file was refined with clearer Do/Don't rules, explicit mapping between `week2_session.b/` (documentation) and `week2_session.a/` (code), and shorter scannable sections.

4 Results

4.1 Exercise 1: AGENTS.md creation

The generated `AGENTS.md` includes project overview, tech stack table, directory structure, key functions (`data_pipeline.py`, `flood_model.py`, `alert_router.py`, etc.), data formats, Open-Meteo endpoint parameters, run commands, and conventions (async I/O, UTC timestamps, WMO-style thresholds). The agent explicitly noted the mismatch between the lab prompt and the real codebase and documented the real implementation.

4.2 Exercise 2: With AGENTS.md

With project context, OpenCode produced `weather_fetcher.py` with `fetch_weather()` and `fetch_all_zones()`, using `aiohttp`, a `WeatherData` dataclass, Open-Meteo, timeouts, and UTC timestamps (Figure 2).

4.3 Exercise 2: Without AGENTS.md

In an empty folder, the first reply only asked for language and file location. After a specification prompt, the agent wrote synchronous code with `requests`, returning a plain `dict` and a NYC demo in `__main__` (Figure 3).

Table 1 summarizes the comparison.

Table 1: Same prompt, different context.

Aspect	Without context	With AGENTS.md
HTTP	<code>requests</code> (sync)	<code>aiohttp</code> (async)
Return type	<code>dict</code>	<code>WeatherData</code> dataclass
Multi-zone	No	<code>fetch_all_zones()</code>
Integration	Standalone demo	Matches flood-warning stack

5 Analysis and validation

AGENTS.md value: The with-context run adopted async patterns and typed models consistent with `AGENTS.md` and `data_pipeline.py`. The without-context run was valid Python but generic; it required a second, more detailed prompt and did not reference zones, stations, or project modules.

Lab vs. implementation: Documenting the real FastAPI/Open-Meteo system is preferable to forcing incorrect Streamlit/OpenWeather text from the handout. The comparison test still satisfies the learning goal: context steers design choices.

Limits: `AGENTS.md` must be maintained as code evolves. Opening the same directory after files exist can prevent a clean “without” retest—an empty folder (or explicit “ignore existing files”) is needed for baseline comparisons.

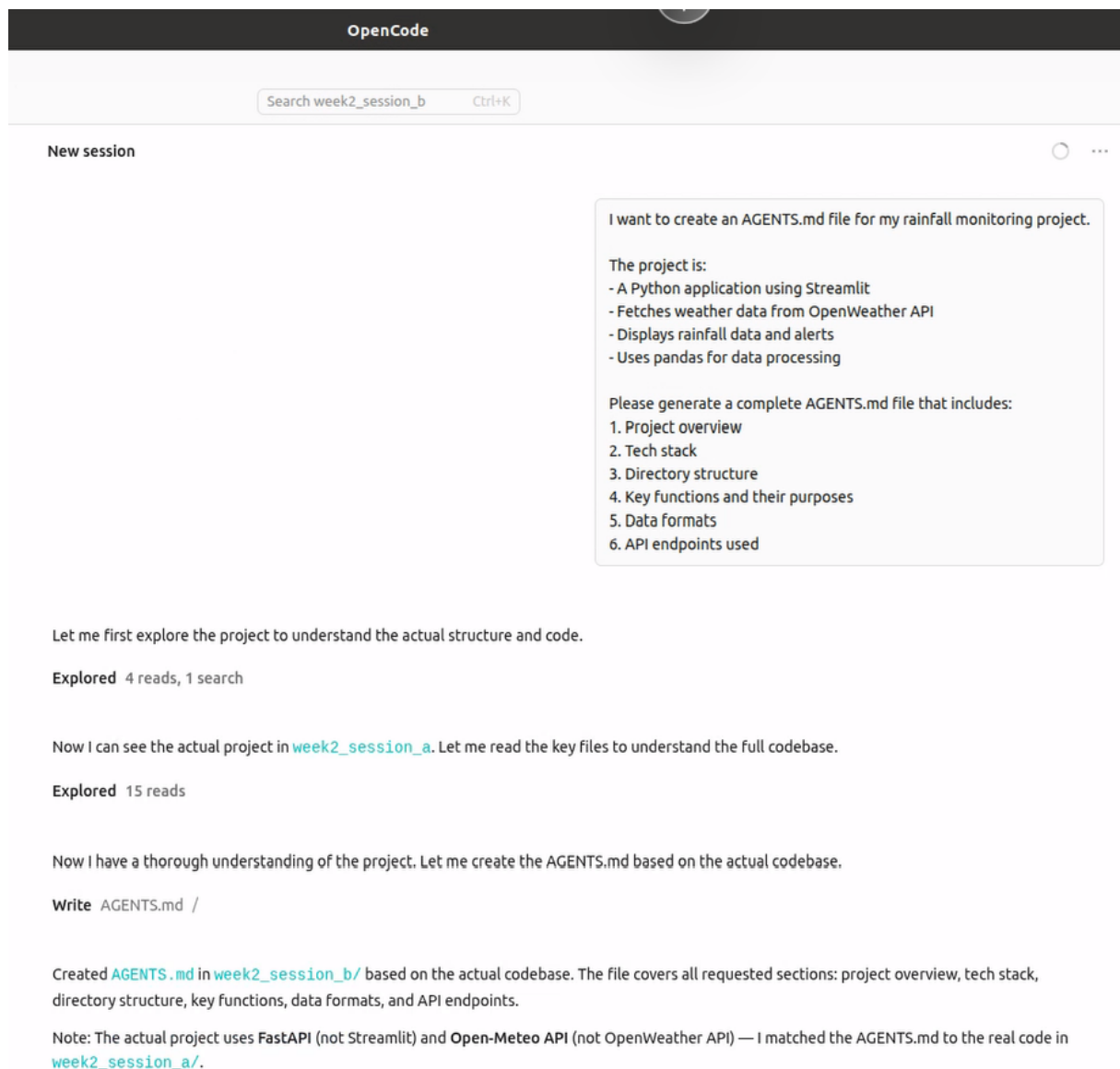


Figure 1: OpenCode: AGENTS.md generation prompt; exploration of `week2_session_a/` and creation of `AGENTS.md`.

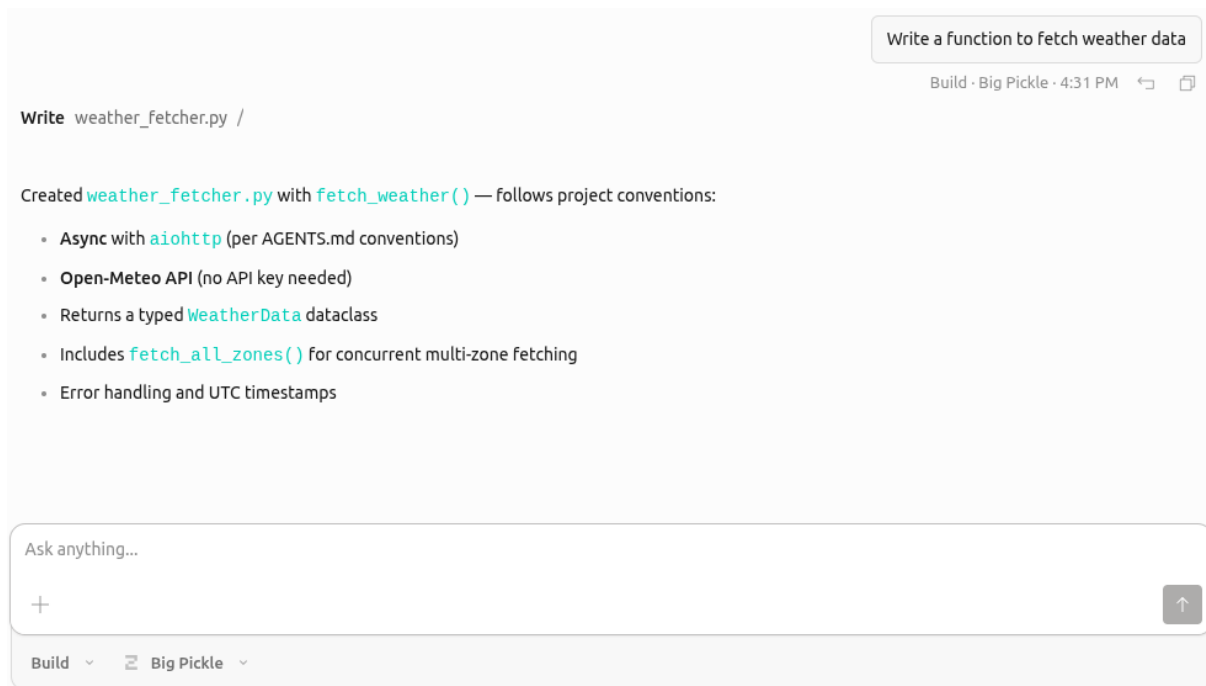


Figure 2: With `AGENTS.md`: “Write a function to fetch weather data” yields `weather_fetcher.py` aligned with project conventions.

6 Discussion

1. Scanning the repo before writing `AGENTS.md` produced more accurate context than trusting the lab prompt alone.
2. The with-context fetcher matched conventions; the without-context version showed that agents default to simpler sync patterns when unstated.
3. Both runs used Open-Meteo; the main gain from context was architecture (async, dataclass, multi-zone), not API choice alone.
4. Future work: add a short Do/Don’t block and link to `docs/` for long examples to save context window space.

7 Problems encountered

A second chat in the project folder only fixed a missing `import asyncio` because `weather_fetcher.py` already existed—not a fresh generation. The empty-folder workflow required two user messages (clarification, then specification) before code appeared.

8 Lessons learned

`AGENTS.md` is project memory for agents: it improved alignment of generated code with the urban flood-warning stack. Comparisons should use a true baseline (empty directory) when possible. Human review remains necessary for imports, API details, and keeping documentation synchronized with code.

9 Conclusion

Week 2 Session B objectives were met: `AGENTS.md` was created from the real codebase, tested with a standard fetch-weather prompt, compared against an empty-folder baseline, and refined

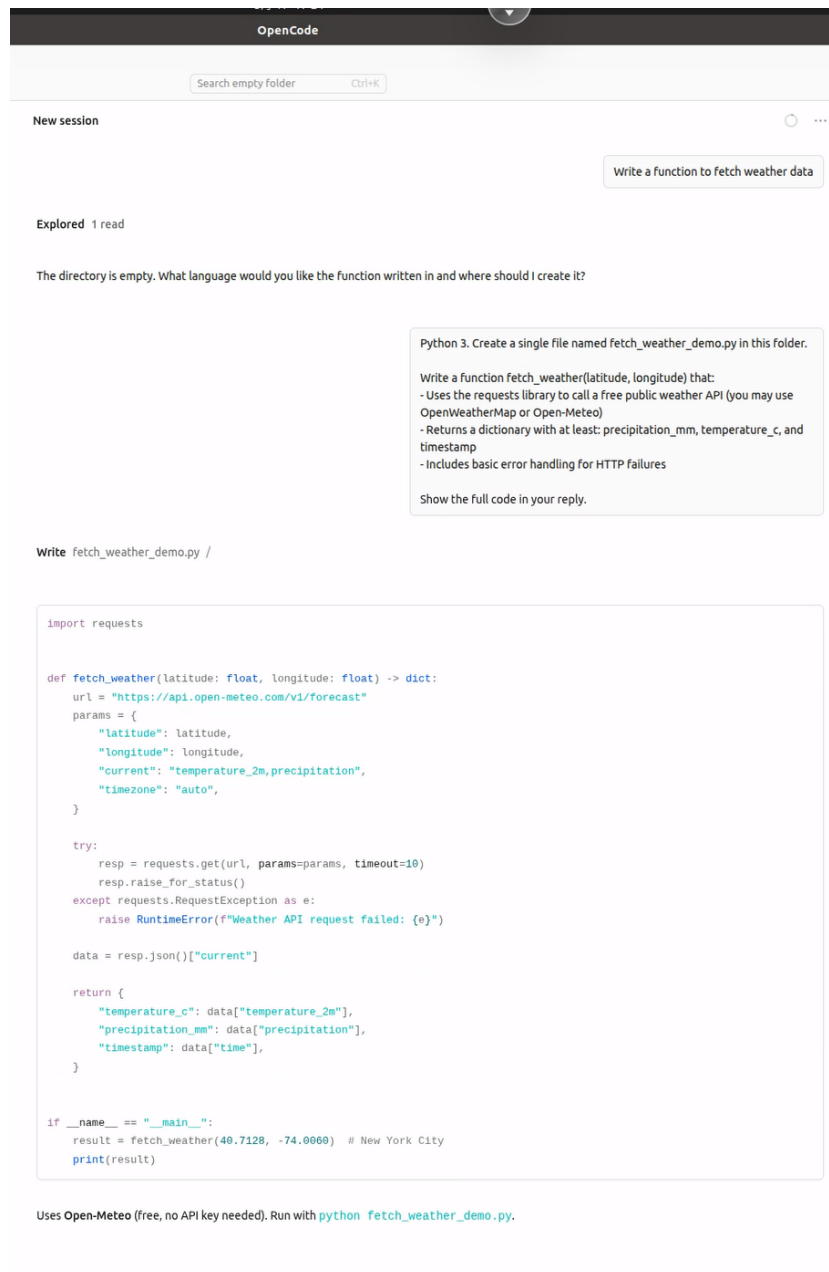


Figure 3: Without project context: empty folder, clarifying exchange, then `fetch_weather_demo.py` (sync requests, dict return).

with conventions and Do/Don't guidance. Submitted artifacts: AGENTS.md, prompt_log.md, weather_fetcher.py (optional), and OpenCode screenshots.

A Submitted prompt log: prompt_log.md

prompt_log.md (full file)

```
# Prompt Log - Week 2 Session B

## Exercise 1: Create AGENTS.md

**Date:** 2026-05-17

**Prompt used:**

I want to create an AGENTS.md file for my rainfall monitoring project.

The project is:
- A Python application using Streamlit
- Fetches weather data from OpenWeather API
- Displays rainfall data and alerts
- Uses pandas for data processing

Please generate a complete AGENTS.md file that includes:
1. Project overview
2. Tech stack
3. Directory structure
4. Key functions and their purposes
5. Data formats
6. API endpoints used

**AI output summary:**

OpenCode explored week2_session_a/ and wrote AGENTS.md documenting the actual codebase:
↳ FastAPI (not Streamlit), Open-Meteo (not OpenWeather), TimescaleDB/Redis, zone-based
↳ flood model, alert router, and real module names (data_pipeline.py, flood_model.py,
↳ etc.). Sections match the lab checklist.

**Edits you made before saving:**

- Matched documentation to real code rather than the lab prompt Streamlit/OpenWeather
↳ wording.
- Saved as week2_session_b/AGENTS.md (documents code in week2_session_a/).
- Verified no API keys in the file; conventions and run commands included.

**Where saved:** ai_water_lab/week2_session_b/AGENTS.md

---

## Exercise 2: Test AGENTS.md effectiveness

**Same question (both tests):** Write a function to fetch weather data

### Without AGENTS.md

**How you ran it:** New OpenCode chat in an empty folder (no AGENTS.md, no existing
↳ project files). Agent first asked for language and path; follow-up requested Python
↳ and fetch_weather_demo.py.

**Summary of AI output:**

- Libraries: requests (synchronous HTTP)
```

- API: Open-Meteo (<https://api.open-meteo.com/v1/forecast>), no API key
- Return type: plain dict with temperature_c, precipitation_mm, timestamp
- Error handling: try/except on requests.RequestException, raises RuntimeError
- Structure: single function + if __name__ == __main__ demo for New York City coordinates
- Not present: async/aihttp, WeatherData dataclass, multi-zone fetch_all_zones,
↳ project-specific station_id or zone config

With AGENTS.md

****How you ran it:**** Project folder open with AGENTS.md and existing week2_session_a/ code
↳ visible to the agent.

****Summary of AI output:****

- File: week2_session_b/weather_fetcher.py
- Libraries: aiohttp, asyncio, dataclass
- API: Open-Meteo with current variables aligned to project needs
- Return type: WeatherData dataclass (station_id, time, precipitation, temp, humidity,
↳ pressure, wind)
- Extra: fetch_all_zones(zones) using asyncio.gather for concurrent multi-zone fetch
- Error handling: per-request timeout, returns None on failure, filters exceptions in
↳ fetch_all_zones
- Follow-up: second chat in same directory only fixed missing import asyncio (file already
↳ existed)

Comparison

Aspect	Without context	With AGENTS.md
HTTP style	Sync requests	Async aiohttp
Data model	dict	WeatherData dataclass
Multi-zone	No	fetch_all_zones()
Project alignment	Generic standalone script	Matches AGENTS.md conventions

****Conclusion:**** AGENTS.md steered the agent toward the project's async stack, typed
↳ models, and Open-Meteo usage patterns. Without context, the answer was still valid
↳ Python but simpler and not integrated with the urban flood-warning architecture.

Exercise 3: Refine AGENTS.md

****What I changed after testing:****

- Clarified that application code lives under week2_session_a/ while AGENTS.md is stored
↳ in week2_session_b/.
- Added Do/Don't guidance: use async I/O and env vars; do not commit .env or API keys; do
↳ not use buggy_rainfall.py in production.
- Linked weather_fetcher.py as an example fetch module consistent with data_pipeline.py
↳ patterns.
- Kept sections scannable (overview, stack, structure, conventions) rather than pasting
↳ full source files.

****Reusable template ideas:****

- Start AGENTS.md from real repo scan, not only the lab prompt.
- Always run a without vs with comparison using an empty folder for the baseline.
- Keep secrets out of AGENTS.md; document env vars only.

Lessons learned

AGENTS.md acts as durable project memory: the with context run produced code aligned with

- ↪ FastAPI/async/Open-Meteo conventions, while the empty-folder run needed clarifying
- ↪ questions and yielded a minimal sync wrapper. Context window limits mean AGENTS.md
- ↪ should stay scannable (overview, stack, structure, conventions) rather than
- ↪ duplicating entire files. Human review still matters (e.g. fixing import asyncio,
- ↪ verifying APIs).